

Variational Autoencoders (VAEs)

A generative model that constrains the latent space to follow a learned distribution — enabling sampling of new data — by combining reconstruction with a KL divergence regulariser and the reparameterization trick to keep the whole pipeline differentiable.

Table of Contents >

- [Manifold Hypothesis](#)
- [Notation](#)
- [From Deterministic to Probabilistic Networks](#)
- [The Differentiability Problem](#)
- [The Reparameterization Trick](#)
- [Why Log Variance](#)
- [ELBO Derivation](#)
 - [Math Prerequisites](#)
 - [Deriving the ELBO](#)
 - [KL Divergence — Closed Form](#)
 - [Reconstruction Loss](#)
 - [Total Loss](#)
- [Architecture](#)
 - [Traditional Autoencoder](#)
 - [VAE Architecture](#)
- [Why Normal for the Latent Space?](#)
- [Implementation](#)
 - [Linear VAE](#)
 - [Convolutional VAE](#)
- [References](#)

Manifold Hypothesis

Most naturally occurring high-dimensional datasets lie on or near a **low-dimensional manifold** embedded in the high-dimensional space.

Example: The space of all possible 256×256 images has $256^2 \times 3 \approx 200,000$ dimensions. But the space of *natural-looking face images* varies along far fewer axes —

pose, lighting, expression, identity. This sub-space is the face manifold.

Why it matters for VAEs: If data lies near a low-dimensional manifold, a compressed latent representation of dimension $d_z \ll d_x$ can capture everything essential about the data. The VAE exploits this by:

1. Encoding high-dimensional inputs to a compact latent space
2. Constraining the latent space to be smooth and continuous (via the KL term)
3. Sampling from that space to generate new data that lies on the same manifold

🔗 The manifold hypothesis is why the decoder can generalise. If you move slightly in latent space, you should land on a nearby point of the manifold — not jump to a completely unrelated image. The KL regularisation is what enforces this smoothness.

Notation

Symbol	Meaning
$p(x)$	True data distribution — unknown and intractable
$p(z)$	Prior on latent variable: $\mathcal{N}(0, I)$
$q(z x)$	Approximate posterior: encoder output $\mathcal{N}(\mu_x, \sigma_x^2)$
$p(x z)$	Likelihood: decoder — probability of reconstructing x given z
θ	Decoder parameters
ϕ	Encoder parameters

From Deterministic to Probabilistic Networks

A standard regression network outputs a single value $\hat{y}$. A **probabilistic** network outputs the *parameters of a distribution* over y :

Old	New
Output: $\hat{y}$ (one scalar)	Output: $(\mu, \log \sigma^2)$ (two scalars)
Loss: MSE	Loss: negative log-likelihood under $\mathcal{N}(\mu, \sigma^2)$

The network learns both the expected value *and* its uncertainty.

🔥 This is the same principle as Bayesian regression — the network learns a posterior over outputs rather than a point estimate.

The Differentiability Problem

Introducing a **probabilistic latent layer** breaks backpropagation. Sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is not differentiable with respect to μ and σ — the stochastic node has no gradient. Training halts at the latent layer.

The Reparameterization Trick

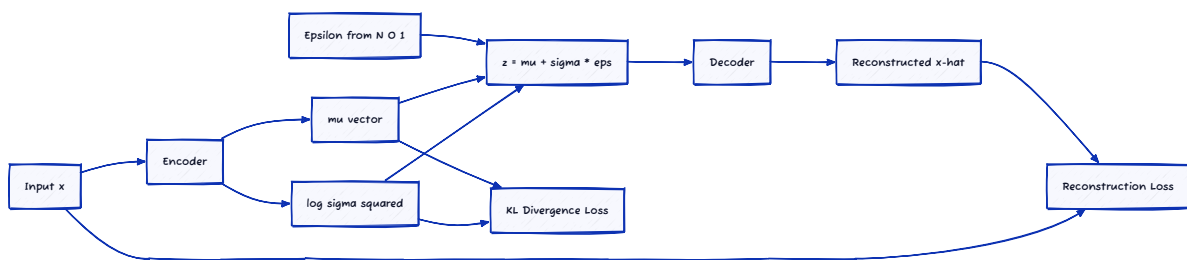
Instead of sampling z directly, write:

$$z = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

Now μ and σ are deterministic (differentiable), and ε is external noise. Gradients flow through μ and σ freely.

Proof that mean and variance are preserved:

$$\mathbb{E}[z] = \mu + \sigma, \mathbb{E}[\varepsilon] = \mu, \quad \text{Var}[z] = \sigma^2, \text{Var}[\varepsilon] = \sigma^2.$$



🔍 Recall >

1. State the reparameterization equation and identify which part carries the gradient.
2. Prove $\text{Var}[z] = \sigma^2$ for $z = \mu + \sigma\varepsilon$.

Why Log Variance

$\sigma^2 > 0$ always — but a raw network output can be any real number. The fix:

Approach	Problem
Clip output	Non-smooth at boundary; zero gradient below
Raw exp	Overflow for large outputs $\rightarrow$ NaN/Inf
Output $\log \sigma^2$, compute $\sigma^2 = \exp(\log \sigma^2)$	Always positive; clamp to prevent overflow

$$\sigma = \exp\left(\frac{1}{2}\log \sigma^2\right), \quad \sigma^2 = \exp(\log \sigma^2).$$

```
log_var = torch.clamp(self.fc_log_var(h), min=-10, max=10) # prevent
overflow
sigma    = torch.exp(0.5 * log_var)
```

⚠ **Raw `exp(output)` without clamping causes NaN/Inf during training when output spikes. Always clamp `log_var` first.**

ELBO Derivation

Math Prerequisites

Tool	Statement
Expectation as integral	$\mathbb{E}_q[f(z)] = \int q(z), f(z), dz$
Log of ratio	$\log(a/b) = \log a - \log b$
Jensen's inequality	For concave log: $\log \mathbb{E}[f(z)] \geq \mathbb{E}[\log f(z)]$
Monte Carlo approximation	$\int f(z), p(z), dz \approx \frac{1}{N} \sum_{i=1}^N f(z_i), z_i \sim p$
Monotone log	$\arg \max P(x) = \arg \max \log P(x)$
KL divergence	$\text{KL}(q p) = \mathbb{E}_q[\log q(z) - \log p(z)] \geq 0$

Deriving the ELBO

Goal: maximise $\log P(x)$ — the log probability of the observed data.

Step 1. Marginalise out the latent variable using the law of total probability:

$$\log P(x) = \log \int p(x|z), p(z), dz.$$

Step 2. Introduce the encoder $q(z|x)$ by multiplying and dividing:

$$= \log \int q(z|x) \cdot \frac{p(x|z)p(z)}{q(z|x)} dz = \log, \mathbb{E}_{q(z|x)} \left[\frac{p(x|z)p(z)}{q(z|x)} \right].$$

Step 3. Apply Jensen's inequality ($\log \mathbb{E}[f] \geq \mathbb{E}[\log f]$):

$$\log P(x) \geq \mathbb{E}_{q(z|x)} \left[\log \frac{p(x|z)p(z)}{q(z|x)} \right].$$

Step 4. Split the logarithm:

$$= \underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{Term A: reconstruction}} + \underbrace{\mathbb{E}_{q(z|x)} \left[\log \frac{p(z)}{q(z|x)} \right]}_{\text{Term B: } -\text{KL}(q|p)}.$$

This lower bound on $\log P(x)$ is the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) || p(z)).$$

Interpretation:

Term	Goal	Interpretation
A — $\mathbb{E}[\log p(xz)]$	Maximise	How close the reconstructed X form the input
B — $\text{KL}(q p)$	Minimise	How far is the encoder posterior from the prior $\mathcal{N}(0, I)$?

Maximising ELBO = maximising A – B = maximising reconstruction while keeping the latent distribution close to the prior.

Monte Carlo approximation: exact ELBO requires integrating over all z . In practice, use a single sample $z \sim q(z|x)$ per training step (the reparameterization trick makes this differentiable):

$$\mathcal{L}_{\text{ELBO}} \approx \log p_\theta(x|z) - \text{KL}(q_\phi(z|x) || p(z)), \quad z = \mu + \sigma \epsilon.$$

🔍 Recall >

1. What step in the ELBO derivation introduces Jensen's inequality, and what condition on $\log$ allows it?
2. What does Term A vs Term B each encourage the network to do?
3. Why is a single Monte Carlo sample sufficient per training step?

KL Divergence — Closed Form

Single-variable case: $q = \mathcal{N}(\mu, \sigma^2)$, $p = \mathcal{N}(0, 1)$.

$$\begin{aligned}\text{KL}(q|p) &= \mathbb{E}_q[\log q(z) - \log p(z)] \\ &= \mathbb{E}_q! \left[-\frac{(z - \mu)^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2) \right] - \mathbb{E}_q! \left[-\frac{z^2}{2} - \frac{1}{2} \log(2\pi) \right]\end{aligned}$$

Using $\mathbb{E}_q[(z - \mu)^2/\sigma^2] = 1$ and $\mathbb{E}_q[z^2] = \sigma^2 + \mu^2$:

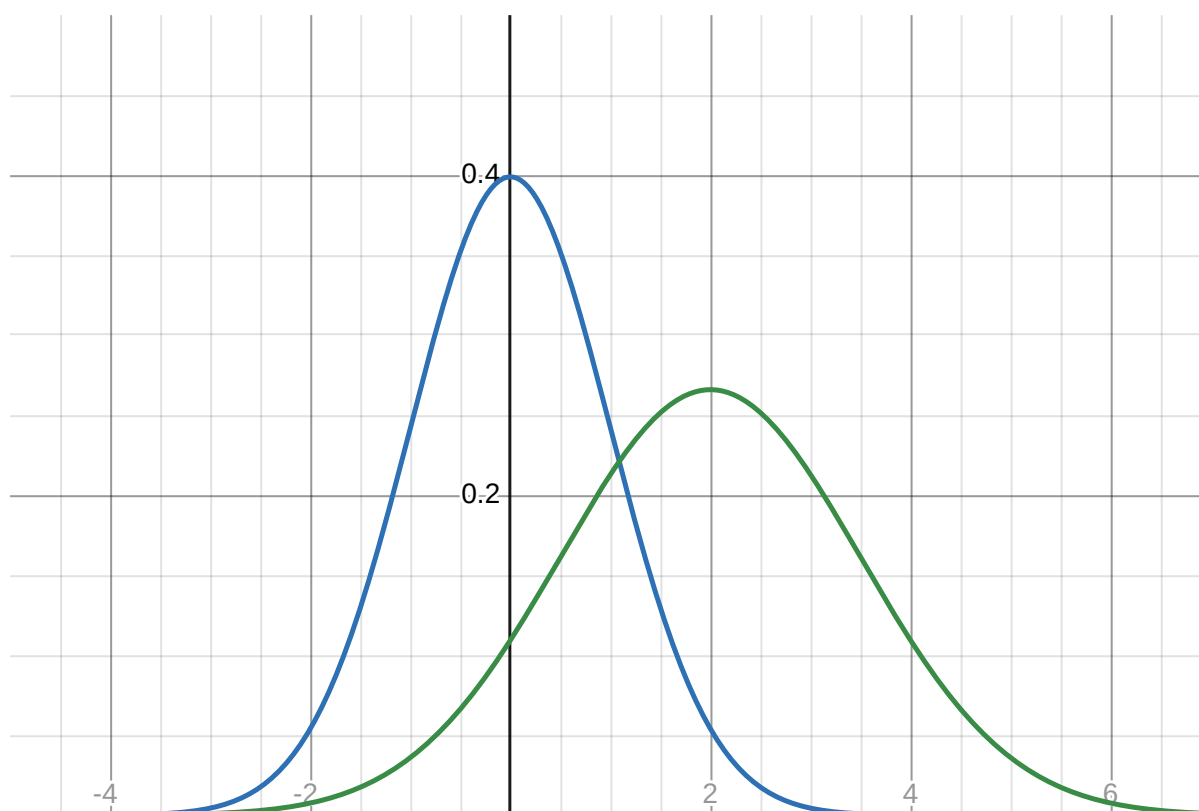
$$= -\frac{1}{2} - \frac{1}{2} \log \sigma^2 + \frac{1}{2}(\sigma^2 + \mu^2) = -\frac{1}{2}(1 + \log \sigma^2 - \mu^2 - \sigma^2). \quad \blacksquare$$

Multivariate case: $q = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\sigma_1^2, \dots, \sigma_d^2))$, $p = \mathcal{N}(\mathbf{0}, I)$.

Because the off-diagonal covariances are zero, the KL factorises over dimensions:

$$\text{KL}(q|p) = \sum_{j=1}^d \text{KL}(\mathcal{N}(\mu_j, \sigma_j^2), \mathcal{N}(0, 1)) = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

Desmos — KL visualised:



$p(z) = \mathcal{N}(0, 1)$ (left) and an unregularised $q(z|x) = \mathcal{N}(2, 1.5^2)$ (right). The KL loss penalises this offset, pulling q back toward p .

Reconstruction Loss

Output type	Loss
Continuous / float pixels	MSE: $\ x - \hat{x}\ ^2$
Binary / normalised pixels $\in [0, 1]$	BCE: $-\sum [x \log \hat{x} + (1 - x) \log(1 - \hat{x})]$

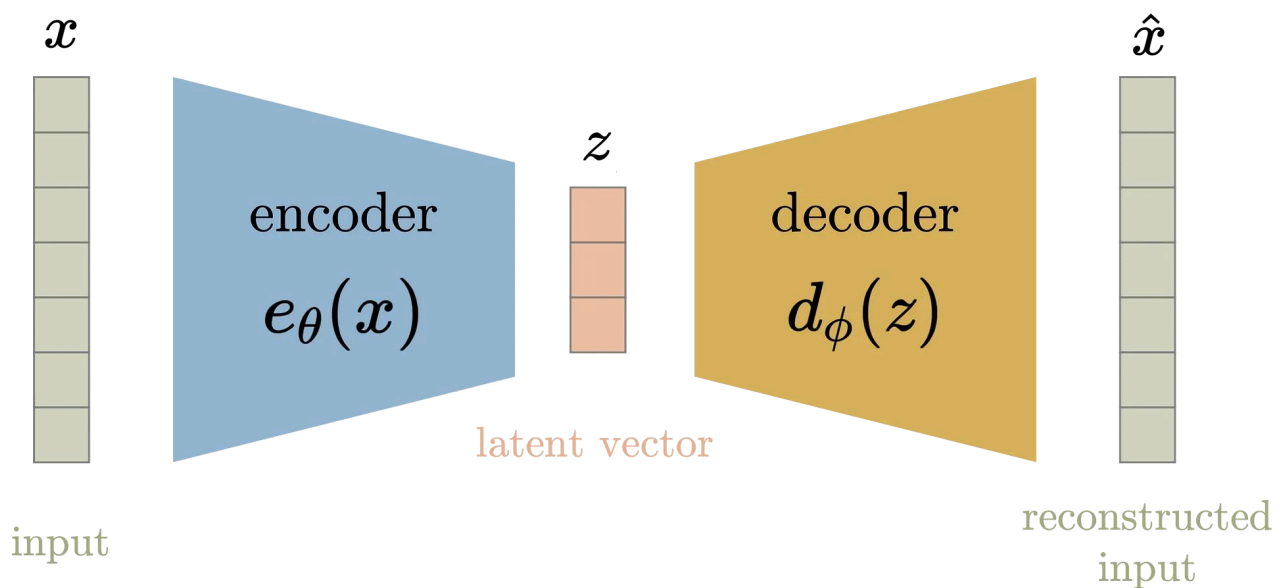
Total Loss

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{recon}}}_{\text{BCE or MSE}} + \underbrace{\mathcal{L}_{\text{KL}}}_{-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)}$$

🔗 β -VAE weights the KL term: $\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta, \mathcal{L}_{\text{KL}}$ with $\beta > 1$. Higher β encourages more disentangled representations at the cost of reconstruction quality.

Architecture

Traditional Autoencoder

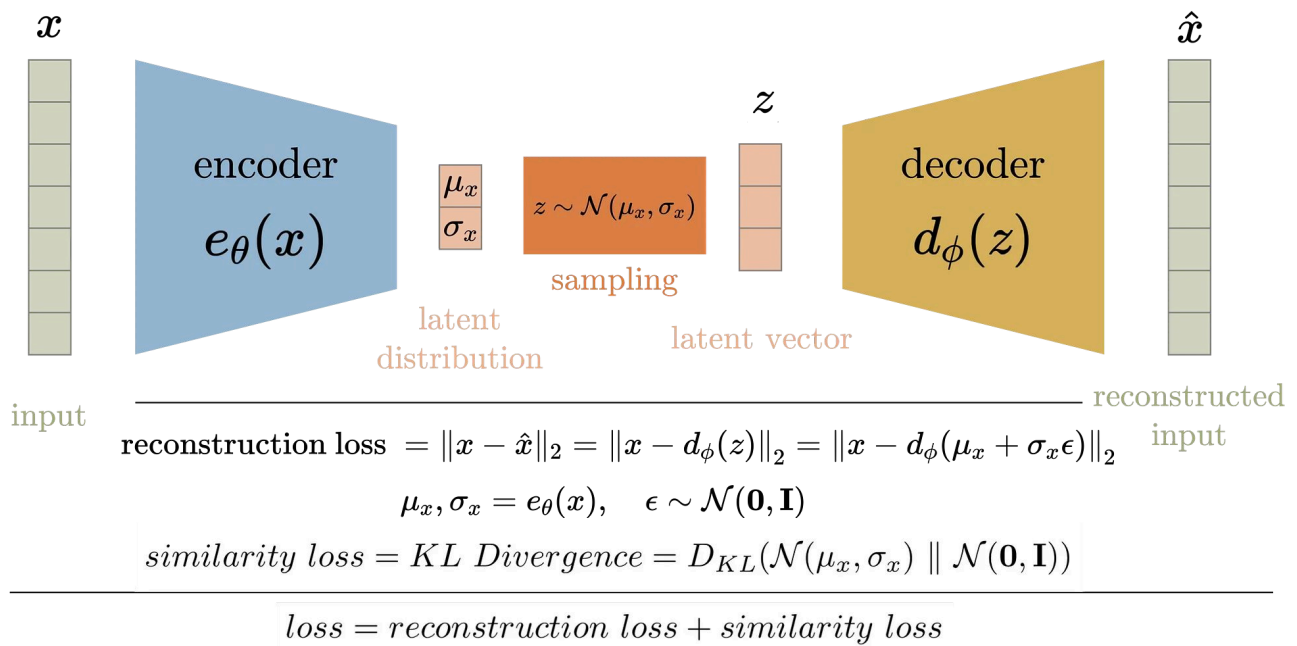


$$\text{loss} = \|x - \hat{x}\|_2 = \|x - d_\phi(z)\|_2 = \|x - d_\phi(e_\theta(x))\|_2$$

Encoder maps x to a fixed point z ; decoder maps z to $\hat{x}$.

Problems for generation: the latent space is unstructured — randomly sampling z produces garbage because there is no prescribed geometry.

VAE Architecture



Encoder outputs $(\mu, \log \sigma^2)$; reparameterized sample z feeds the decoder.

Component	AE	VAE
Encoder output	z — one vector	$\mu, \log \sigma^2$ — two vectors
Latent sample	Deterministic	$z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(0, I)$
Losses	Reconstruction only	Reconstruction + KL
Generation	Fails	Sample $z \sim \mathcal{N}(0, I) \rightarrow$ decoder $\rightarrow$ new data

🔗 Recall >

1. Name the two encoder output heads and what each represents.
2. Why can a VAE generate new data while a traditional AE cannot?
3. What does the KL term enforce geometrically?

Why Normal for the Latent Space?

By the **Universal Approximation Theorem**, a deep decoder with nonlinear activations can map $\mathcal{N}(0, I)$ samples to any data distribution. We choose normal because:

- KL against $\mathcal{N}(0, 1)$ has a closed form — no numerical integration
- Easy to sample from at inference (no encoder needed)
- Isotropic — no preferred direction in latent space

At inference: draw $z \sim \mathcal{N}(0, I)$, pass through decoder — encoder is not involved.

Implementation

Linear VAE

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int, latent_dim: int):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(input_dim, hidden_dim),
nn.ReLU())
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_log_var = nn.Linear(hidden_dim, latent_dim) # log  $\sigma^2$ ,
not  $\sigma^2$ 
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, input_dim), nn.Sigmoid(),
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        log_var = torch.clamp(self.fc_log_var(h), min=-10, max=10)
        return mu, log_var

    def reparameterize(self, mu, log_var):
        if not self.training:
            return mu # deterministic at
inference
        return mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)

    def forward(self, x):
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        return self.decoder(z), mu, log_var

    @staticmethod
    def loss(x, x_recon, mu, log_var):
        recon = F.binary_cross_entropy(x_recon, x, reduction='sum')
        kl = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
        return recon + kl

def generate(model, n, latent_dim):
    z = torch.randn(n, latent_dim) # sample prior – no encoder
    with torch.no_grad():
        return model.decoder(z)
```

Convolutional VAE

For image data (e.g. CIFAR-10 at $32 \times 32 \times 3$). With kernel=2, stride=2, padding=1, each conv halves the spatial dimension:

$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \implies 512 \times 2 \times 2 = 2048$ features before $\mu / \log \sigma^2$.

⚠ The decoder receives a 1D latent vector but ConvTranspose2d expects a 4D tensor. A linear projection + reshape layer is required between the latent code and the first ConvTranspose2d — missing this is a common implementation bug.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.datasets import CIFAR10, CIFAR100
import torchvision.transforms as T

class ConvVAE(nn.Module):
    def __init__(self, latent_size: int = 200):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=2, stride=2, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(True), nn.Dropout(0.2),
            nn.Conv2d(32, 64, kernel_size=2, stride=2, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(True), nn.Dropout(0.2),
            nn.Conv2d(64, 128, kernel_size=2, stride=2, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(True), nn.Dropout(0.2),
            nn.Conv2d(128, 256, kernel_size=2, stride=2, padding=1),
            nn.BatchNorm2d(256), nn.ReLU(True), nn.Dropout(0.2),
            nn.Conv2d(256, 512, kernel_size=2, stride=2, padding=1),
            nn.BatchNorm2d(512), nn.ReLU(True), nn.Dropout(0.2),
            nn.Flatten(),
            nn.Linear(2048, 512), # 512 * 2 * 2 = 2048 for 32x32 input
        )
        self.fc_mu = nn.Linear(512, latent_size)
        self.fc_logvar = nn.Linear(512, latent_size)

        # Project latent vector back to spatial tensor before
        # ConvTranspose layers
        self.decoder_input = nn.Sequential(
            nn.Linear(latent_size, 512),
            nn.ReLU(True),
            nn.Linear(512, 2048), # will be reshaped to (B, 512, 2, 2)
            nn.ReLU(True),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2,
padding=1), nn.BatchNorm2d(256), nn.ReLU(True), nn.Dropout(0.2),
            nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2,
padding=1), nn.BatchNorm2d(128), nn.ReLU(True), nn.Dropout(0.2),
            nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2,
padding=1), nn.BatchNorm2d(64), nn.ReLU(True), nn.Dropout(0.2),
            nn.ConvTranspose2d(64, 32, kernel_size=2, stride=2,
padding=1), nn.BatchNorm2d(32), nn.ReLU(True), nn.Dropout(0.2),
```

```

        nn.ConvTranspose2d(32, 3, kernel_size=2, stride=2,
padding=1),
        nn.Sigmoid(),
    )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        log_var = torch.clamp(self.fc_logvar(h), min=-10, max=10)
        return mu, log_var

    def reparameterize(self, mu, log_var):
        if not self.training:
            return mu
        return mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)

    def forward(self, x):
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        h = self.decoder_input(z).view(-1, 512, 2, 2) # reshape
to spatial
        return self.decoder(h), mu, log_var

    @staticmethod
    def loss(x, recon_x, mu, log_var):
        recon = F.binary_cross_entropy(recon_x, x, reduction='sum')
        kl = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
        return recon + kl

# Training
transform = T.Compose([T.ToTensor()])
dataset = torch.utils.data.ConcatDataset([
    CIFAR10(root='./data', download=True, transform=transform),
    CIFAR100(root='./data', download=True, transform=transform),
])
loader = torch.utils.data.DataLoader(dataset, batch_size=32,
shuffle=True)
model = ConvVAE(latent_size=200)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(100):
    for x, _ in loader:
        optimizer.zero_grad()
        recon_x, mu, log_var = model(x)
        loss = ConvVAE.loss(x, recon_x, mu, log_var)
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch+1:3d}/100 Loss: {loss.item():.2f}")

```

References

- Variational Autoencoders from scratch

- Gumbel-Softmax (discrete reparameterization): sassafra13.github.io/GumbelSoftmax
 - Kingma & Welling, *Auto-Encoding Variational Bayes*, 2013.
-

Tags: [#deep-learning/vae](#) [#ml/generative-models](#) [#ml/probabilistic-modeling](#)
[#ml/reparameterization](#) [#ml/elbo](#)